

Benchmarking Classical Quantum Circuit Simulators

Hevish Cowlessur

In this document, we compare runtime and approximate memory usage as qubit count and circuit depth increase for a representative VQC and determine which backend is most suitable for different practical scenarios.

Objective. This study benchmarks four CPU-based exact state-vector simulators for a representative variational quantum circuit (VQC) workload: Qiskit `Aer (statevector)`, PennyLane `lightning.qubit`, Qulacs, and Google Cirq’s `qsim` extension `qsimcirq`.

Rationale. The four simulators were selected based on four criteria: they all support CPU-based exact state-vector simulation, they are relevant to parametric/VQC workloads, they represent major quantum software ecosystems, and they are mature enough to make the benchmark practically meaningful.

Methodology and Experiment Configurations. All simulators are evaluated on the same family of circuits, using identical qubit counts (q), circuit depths (d), and parameter tensors generated from fixed random seeds. Each circuit consists of repeated layers of R_Y and R_Z rotations on every qubit, followed by a ring of CNOT gates. We benchmark $q \in \{2, 4, \dots, 28\}$ and $d \in \{2, 4, 6\}$. To keep the comparison controlled, all benchmarks are run on CPU only and use the same constant-size observable output, $\langle Z_0 \rangle$, rather than a full statevector or probability distribution. One untimed warm-up execution is used to reduce incidental first-run effects, after which five timed runs are averaged. The primary reported metrics are wall-clock runtime and approximate peak memory increase, with a 60s timeout used as an operational indicator of scalability. Full implementation details are provided in the accompanying notebook.

Together, this ensures the comparison reflects simulator execution cost while minimizing backend-specific output overhead.

Benchmarking Results Summary. Detailed plots and tables are provided in the accompanying notebook. Overall, qubit count is the dominant driver of runtime and practical scalability, while depth acts as a secondary scaling factor. Across the tested range, `qsim` is the strongest runtime performer overall, followed by Qiskit `Aer`. Qulacs is competitive at smaller sizes but shows much stronger memory growth in the high-qubit regime. PennyLane `lightning.qubit` is consistently slower, with rapid memory growth in high-qubit regimes and less scalable in this CPU-only exact-simulation setting.

Which simulator is best for which scenario?

- **Small-scale/fast-iteration regime:** `qsim` is the preferred backend, with Qulacs and PennyLane `lightning.qubit` still acceptable at low qubit counts where memory pressure is not yet limiting. Qiskit `Aer` is also a reasonable second choice at low qubit counts.
- **Larger exact-simulation regime:** `qsim` is the strongest choice, since it combines the best runtime with no comparable memory growth in the tested range.
- **Balanced secondary option:** Qiskit `Aer` is the most reasonable fallback when ecosystem familiarity and overall balanced behaviour are important.
- **Least suitable for larger workloads:** Qulacs and PennyLane `lightning.qubit` show the earliest practical degradation in the upper-qubit regime, primarily due to stronger memory growth.

Scalability limits. Within the tested range, Qulacs and PennyLane `lightning.qubit` show the earliest practical scalability limits, primarily through stronger memory growth in the upper-qubit regime. By contrast, `qsim` and Qiskit `Aer` remain usable furthest into the tested range, with `qsim` retaining the strongest runtime headroom under the most demanding cases.

Recommendation from the overall summary analysis. For **fast development and iteration**, defined here as the low-qubit regime $q \leq 10$, the preferred backend is `qsim`, since it gives the best runtime while remaining fully reliable. For **precise computations at higher qubit counts**, defined here as $q \geq 20$, the preferred backend is again `qsim`, since it provides the best overall runtime-memory trade-off among the exact simulators tested. Qiskit `Aer` is the strongest secondary choice, especially where Qiskit ecosystem compatibility is important.

Limitation. This study is CPU-only and compares exact state-vector simulators only; the conclusions therefore do not directly extend to GPU-enabled or tensor-network simulation backends. Memory was measured using process-level peak RSS delta in fresh subprocesses; this is useful for practical comparison, but it may not fully capture allocator-level or backend-internal memory usage.